

GESTIONE DELLE PRIORITÀ

Abbiamo visto che la gestione di priorità più semplice sfrutta il **DAISY CHAIN**, quindi c'è un concetto di vicinanza verso il processore, i dispositivi fisicamente più vicini alla CPU hanno priorità maggiore.

Quando attiviamo l'esecuzione di un driver, le INTERRUZIONI devono essere mascherate, e in generale fin ora la gestione delle priorità non è ben fatta!

PER VARI MOTIVI:

1. RISCHIO DI PRIORITY INVERSION

IMMAGINIAMO CHE NELLA CATENA A DAISY CHAIN È IL DISPOSITIVO PIÙ LONTANO CHE ALZA LA BANDIERINA che vuole che venga servita la sua richiesta di INTERRUZIONE.

IL PROCESSORE RICEVE QUESTA RICHIESTA DI INTERRUZIONE E ABILITA L'ESECUZIONE DI QUEL DRIVER; E QUEL DRIVER PRENDE IL CONTROLLO FIN TANTO CHE NON TERMINA.

COSA SUCCEDEREBBE SE ALTRI DISPOSITIVI PIÙ VICINI NEL DAISY CHAIN RICHIEDONO L'ATTENZIONE DEL PROCESSORE? SI INVERTE.

2. RISCHIO DI STARVATION

I DISPOSITIVI IN DAISY CHAIN VENGONO SERVITI IN ORDINE DI PRIORITÀ DALLA CPU, I DISPOSITIVI PIÙ IN FONDO NEL DAISY CHAIN NON HANNO PIÙ POSSIBILITÀ DI ESSERE ESEGUITI.

NEL GESTIRE LE PRIORITÀ, IO VORREI INNANZITUTTO ELIMINARE L'INVERSIONE DI PRIORITÀ.

VORREI CHE CI FOSSE UNA GESTIONE DELLE PRIORITÀ BASATA SU UN DISPOSITIVO DI ALTA PRIORITÀ DI POTER INTERRUPIRE L'ESECUZIONE DI ATTIVITÀ A PRIORITÀ PIÙ BASSA.

QUINDI LE ATTIVITÀ A PRIORITÀ BASSA POSSONO ESSERE INTERRUTE DA ATTIVITÀ A PRIORITÀ ALTA.

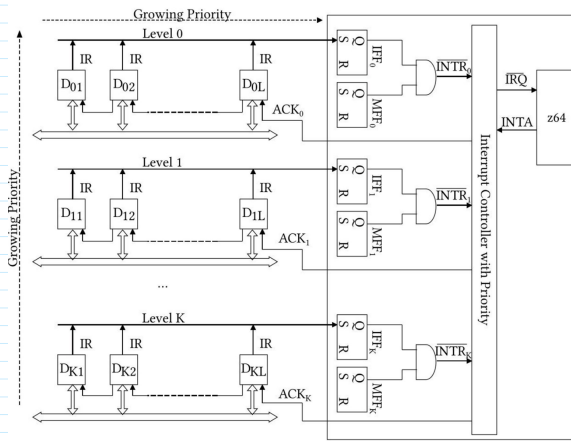
CIÒ VUOL DIRE CHE IL DAISY CHAIN NON MI BASTA!

BISOGNA IMPLEMENTARE DEI LIVELLI DI PRIORITÀ.

IL SEGNALE DI IR CHE VIENE RICEVUTO DALLA CPU, NON È UN SINGOLO SEGNALE CHE PROVIENE DA UN DAISY CHAIN, MA È UN SEGNALE CHE PROVIENE DA UN INSIEME DI DAISY CHAIN.

CI POSSONO ESSERE MOLTE ORGANIZZAZIONI A DAISY CHAIN.

Controllore degli interrupt a livelli di priorità



- Si possono utilizzare più daisy chain, ciascuna associata a un livello di priorità
- I flip-flop di maschera permettono di gestire i livelli di priorità

Possiamo raggruppare i dispositivi su canali di interruzione di richieste differenti, e su ciascun canale ci sono le singole richieste di interruzione che provengono da una singola catena in daisy chain, inserita nel controllore delle interruzioni. Cos'è questo controllore?

È una rete combinatoria che manda la richiesta di interruzione (IRQ) verso

il processore, nel caso in cui provenga una richiesta di interruzione da parte di una catena di daisy chain che ha priorità potenzialmente maggiore rispetto ad una catena di daisy chain che ha già inviato una richiesta di interruzione.

Se per esempio il mio processore sta servendo una richiesta di interruzione di un livello k inferiore, se arriva la richiesta del livello 0, il controllore delle interruzioni dovrà dare il controllo alla richiesta di interruzione del livello 0 (priorità più alta) inviando come IRQ una nuova richiesta di interruzione al processore.

Se c'è una richiesta di interruzione che stiamo servendo sul livello 0, e arriva una richiesta dal livello k , questa ultima richiesta non deve essere mandata al processore!

Quindi organizziamo i nostri canali di interruzione su più livelli.

I livelli più alti hanno la priorità su quelli più bassi.

Come evito il problema di starvation fra questi canali?

Sono presenti flip-flop MFF che sono dei flip flop di maschera e servono a permettere di silenziare le richieste di interruzione che provengono da uno specifico livello.

"Per un certo lasso di tempo, tutte le richieste provenienti dal livello 0, devono essere silenziate" e quindi è necessario resettare il flip-flop MFF 0, così la richiesta di interruzione dal livello zero non viene ricevuta dal processore.

Il processore può ricevere le richieste di interruzione dai livelli sottostanti!

Il singolo livello viene gestito in daisy chain.

Ricapitolando...

Abbiamo visto che per permettere ad un processore di trasferire dati al dispositivo, questo può essere fatto o con un protocollo basato su firmware, ma è poco inefficiente, e si utilizza un'interazione di tipo I/O programmato, quindi è il software che governa l'interazione tramite istruzioni di input dedicate.

Immaginiamo di dover trasferire file di 1 kilobyte dal disco a memoria, il controllore del disco alzerà una richiesta di interruzione verso il processore, il processore avvia il driver, il driver recupera i dati e li scrive in memoria.

Il processore avendo dati può leggere dal dispositivo?

per una richiesta di trasferimento verso il processore, il processore chiama il driver, il driver recupera i dati e li scrive in memoria.

Il processore quanti dati può leggere dal dispositivo?
qual'è la dimensione del data bus verso i dispositivi di I/O? 8 byte.

quindi generiamo un gran numero di richieste di interruzione e di esecuzione di driver, per trasferire dati da disco a memoria.

Avviene spesso che io debba trasferire dati da disco a memoria, ha una frequenza molto alta, perché ciò avviene ogni volta che lanciamo un programma.

Immaginiamo se facendo doppio click su un documento, ogni volta portiamo 10 richieste di interruzione per trasferire dati dal disco alla memoria;

IL SISTEMA RALLENTEREBBE DRASTICAMENTE.

LA SOLUZIONE PREVEDE UN CANALE DEDICATO PER IL TRASFERIMENTO DATI DAI DISPOSITIVI A MEMORIA.

Esso prevede l'utilizzo di un dispositivo dedicato ESTERNO AL PROCESSORE che si occupa di fare il trasferimento di dati da periferica a memoria, SENZA DISTURBARE IL PROCESSORE.

È un componente che viene programmato dal processore, il cui compito è farsi carico del trasferimento dati dai dispositivi a memoria.

Tutto quello che abbiamo visto sin ora, prende il nome di **MEMORIA AD ACCESSO DIRETTO**, ogni volta che devo trasferire dei dati da un disco (ad esempio) alla memoria, devo **PASSARE PER IL PROCESSORE.**

Il processore deve interagire in maniera **SINCRONA** o **ASINCRONA** con il **CONTROLLORE** del disco, leggere 4,8 byte, e copiarli in memoria.

IL CONTROLLORE DEL DISCO NON HA ACCESSO DIRETTO ALLA MEMORIA.

Ecco perché con l'aggiunta di questo componente forniamo un canale dedicato dal dispositivo alla memoria.

UNA LINEA DI COMUNICAZIONE DIRETTA !

QUESTO COMPONENTE STA IN MEZZO E PRENDE IL NOME DI :



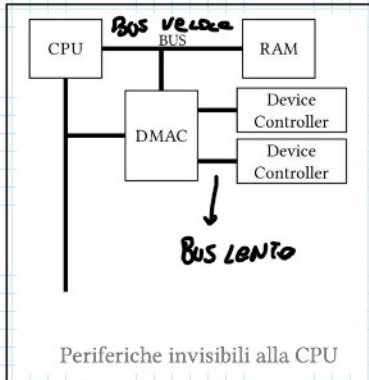
il cui obiettivo è quello di parlare con il processore che permette di fare trasferimento da dispositivo a memoria senza dover interrompere l'esecuzione del mio processore.

ARCHITETTURE DIFFERENTI POSSONO PREVEDERE ORGANIZZAZIONI DIFFERENTI DEL DMAC.

- Il Direct Memory Access Controller (DMAC) è un controllore di bus (coprocessore) che effettua trasferimenti tra dispositivi e memoria al posto della CPU

Organizzazione del DMAC

1. PERIFERICHE INVISIBILI AL PROCESSORE.



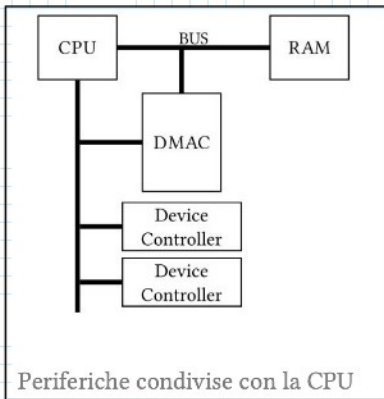
IL DMAC AVRA' dei BUS verso i dispositivi.

Esso È l'UNICO ATTORE del SISTEMA che INTERAGISCE con questi DISPOSITIVI.

IL PROCESSORE deve necessariamente programmare IL DMAC PER accedere a questi dispositivi ed effettuare IL TRASFERIMENTO dati!

IN QUESTO CASO, IL DMAC PRENDE IL Nome di **Adattatore di Bus!**

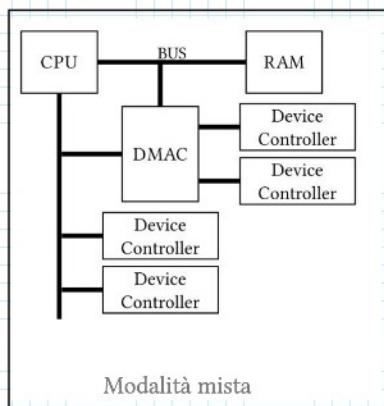
2. PERIFERICHE CONDIVISE CON LA CPU.



I Dispositivi sono SIA VISIBILI AL PROCESSORE TRAMITE L' I/O BUS, SIA VISIBILI AL DMAC TRAMITE L' I/O BUS.

QUINDI IL PROCESSORE può decidere di INTERAGIRE con i dispositivi, PER RICEVERE dati tramite le INTERRUZIONI / BUS WAITING, etc... oppure può decidere di chiedere al DMAC di fare IL TRASFERIMENTO!

3. MODALITÀ MISTA



Alcuni dispositivi si trovano nello stesso BUS dei dispositivi, magari quelli più veloci, e gli altri dispositivi, magari quelli più lenti, si trovano sul DMAC.

TIPICAMENTE QUESTA È L'ORGANIZZAZIONE CHE C'È NELLE ARCHITETTURE MODERNE.

NORTH BRIDGE E SOUTH BRIDGE, SONO FORME PIÙ COMPLESSE DI QUESTO;

• COME ORGANIZZARE IL NOSTRO DMAC !

Esso si comporta da coprocessore per dispositivi messi sul bus dedicato suo o su quello condiviso. E visto come una periferica da parte del processore, quest'ultimo deve poter parlare col DMAC come se fosse una periferica di output.

Esso si comporta da coprocessore per dispositivi messi sul BUS dedicato suo o su quello condiviso. E visto come una periferica da parte del processore, quest'ultimo deve poter parlare col DMAC come se fosse una periferica di output.

IL DMAC E LA CPU condividono il BUS verso la memoria!

MA se ha più dispositivi che scrivono dati sullo stesso BUS contemporaneamente non va bene, quindi significa che dobbiamo inventarci qualche meccanismo per implementare un coordinamento efficiente tra DMAC e processore per garantire che uno solo dei due alla volta, utilizzi il BUS.

Dobbiamo progettare un protocollo di handshaking dedicato fra processore e DMAC per consentire un **ARBITRAGGIO DEL BUS**, ovvero un sistema che determini chi dei due può avere accesso al BUS.

IL DMAC A questo punto diventa uno dei dispositivi più importanti del nostro sistema e lo trattiamo in maniera speciale!

→ È in grado di sganciare il processore da alcuni costi;

È in grado di mandare segnali di controllo verso la memoria, per permettere a quest'ultima di leggere o scrivere dati, provenienti dai dispositivi.

Bisogna prevedere altri due segnali di controllo dedicati all'interfacciamento tra DMAC e processore.

IL PRIMO SEGNALE DI CONTROLLO LO CHIAMIAMO MBR (Memory Bus Request)

IL DMAC COMUNICA AL PROCESSORE CHE VORREBBE USARE IL BUS. (IL BUS VERSO la memoria).

PER fare un TRASFERIMENTO dati, ha letto 64 bit da disco e li vuole scrivere da qualche parte, e chiede il permesso al processore.

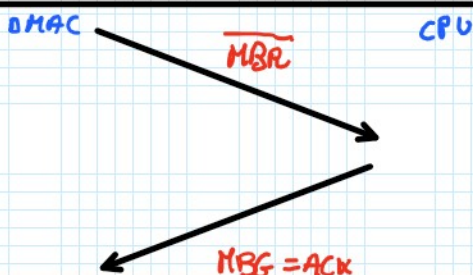
NEL FRATTEMPO IL DMAC **ASPETTA ...** ASPETTA che il **PROCESSORE** gli dia **L'OK!**

↓
E che quindi gli mandi un segnale di Acknowledge.

↓
OK, puoi fare il TRASFERIMENTO dati.

Questo segnale prende il nome di MBG.

- È necessario realizzare un protocollo di handshaking dedicato
 - Memory Bus Request (MBR): allerta la CPU della volontà di effettuare un trasferimento dati da dispositivo a memoria
 - Memory Bus Grant (MBG): il processore rilascia il bus (alta impedenza) per permettere il trasferimento da parte del DMAC
- Fintanto che il segnale $\overline{\text{MBR}}$ è asserted, la CPU è in stallo



A quel punto posso scrivere e leggere dati da/verso la memoria!

A quel punto possa scrivere e leggere dati da/verso la memoria!

QUANDO PUÒ RIPRENDERE IL CONTROLLO IL PROCESSORE? fin tanto che è asserted MBR ossia, il processore è in stallo.

- Cosa succede se il processore vuole effettuare un trasferimento dati mentre il DMAC sta trasferendo dati?

BUS STEALING

IL DMAC chiede il permesso di usare il BUS, fa il trasferimento e rilascia il BUS. IL PROCESSORE VA AVANTI.

DMAC e CPU si rubano a vicenda continuamente l'utilizzo del memory bus.

NON BISOGNA PIÙ MANTENERE IL MBR fino al termine di tutto il trasferimento, l'UNITÀ di controllo del DMAC fa una richiesta del BUS, trasferisce i dati e rilascia il BUS.

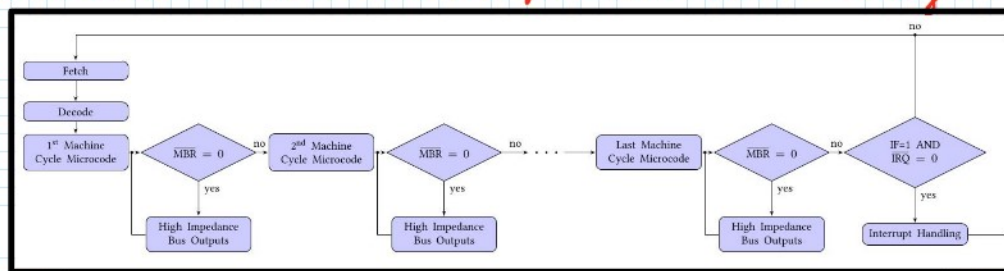
IL FUNZIONAMENTO DEL NOSTRO CICLO ISTRUZIONI CAMBIA ANCORA. $\hookrightarrow$ IL SINGOLO DATO AUMENTARE.

- LA CPU ESEGUE LA FASE DI FETCH (SI PRENDE L'ISTRUZIONE)
- ESEGUE IL DECODE ED ESEGUE IL MICROCODE DEL PRIMO CICLO MACCHINA
- si chiede se il DMAC ha chiesto il BUS:

- SE SÌ, SI DEVE MANTENERE IN ALTA IMPEDENZA E CILIA fin tanto che il MBR non è ancora asserted, permangono in questo ciclo per poco tempo la CPU, e poi va avanti nell'esecuzione.

Dopo aver eseguito un altro ciclo si chiede se c'è una richiesta da parte del DMAC
se lo, prosegue...

SE IL DMAC NON CHIEDE L'UTILIZZO DEL BUS, IL PROCESSORE CONTINUA ad eseguire le NOSTRE ISTRUZIONI.



- Burst mode:** quando il processore asserisce MBR, il DMAC mantiene MBR fino al termine del batch di trasferimento
 - Sottoutilizzo del bus di sistema
 - Il DMAC passa la maggior parte del tempo ad interagire con le periferiche
 - Può essere utile se la CPU non può proseguire con le sue attività se non ha completato l'acquisizione dei dati (situazione rara)
- Bus stealing:** il DMAC richiede il bus solo quando ha un dato pronto da trasferire

NELLE ARCHITETTURE moderne IL DMAC VIAGGIA esattamente alla velocità del processore.

SE IL DMAC NON CHIEDE L'USO DEL BUS, LA CPU CONTINUA ad eseguire la VOSTRA ISTRUZIONE!

Viene interrotto soltanto nel momento in cui il DMAC ha un dato pronto da trasferire.

ALLA FINE DI TUTTI QUESTI CICLI, IL NOSTRO PROCESSORE DEVE ANCHE CHIEDERSI SE CI SONO RICHIESTE DI INTERRUZIONE, SE CI SONO RICHIESTE DI INTERRUZIONE ALLORA ATTIVA L'ESECUZIONE DEL DRIVER, SE NON CI SONO SI RIPARTE CON IL CICLO DELLA FASE DI FETCH.

IN UN PROCESSORE MULTICORE, L'ESECUZIONE DELLA NOSTRA ISTRUZIONE VIENE SPEZZATA, E TRA UNA MICROOPERAZIONE E L'ALTRA PUO' succedere qualcosa, ASSIA IL PROCESSORE SI FERMARE E DA ACCESSO AL BUS AL DMAC.

QUESTO E' IL BUS STEALING!

COME PUO' IL PROCESSORE INTERAGIRE COL DMAC?

RICORDIAMO CHE IL DMAC E' VISTO COME UN DEVICE DEL PROCESSORE!

TRA IL DMAC E LA CPU DOBBIAMO METTERE, SICCOME IL DMAC E' SULL' I/O BUS, UN'INTERFACCIA!

L'I/O BUS FUNZIONA TRAMITE INTERFACCIA, QUINDI, TUTTI I DISPOSITIVI DEVONO ESSERE ASTRATTI TRAMITE UN'INTERFACCIA.

COSA DEVE FARE IL PROCESSORE PER PROGRAMMARE UN TRASFERIMENTO?

INNANZITUTTO DOBBIAMO DIRE QUAL'E' IL DISPOSITIVO DA CUI PRENDERE I DATI.

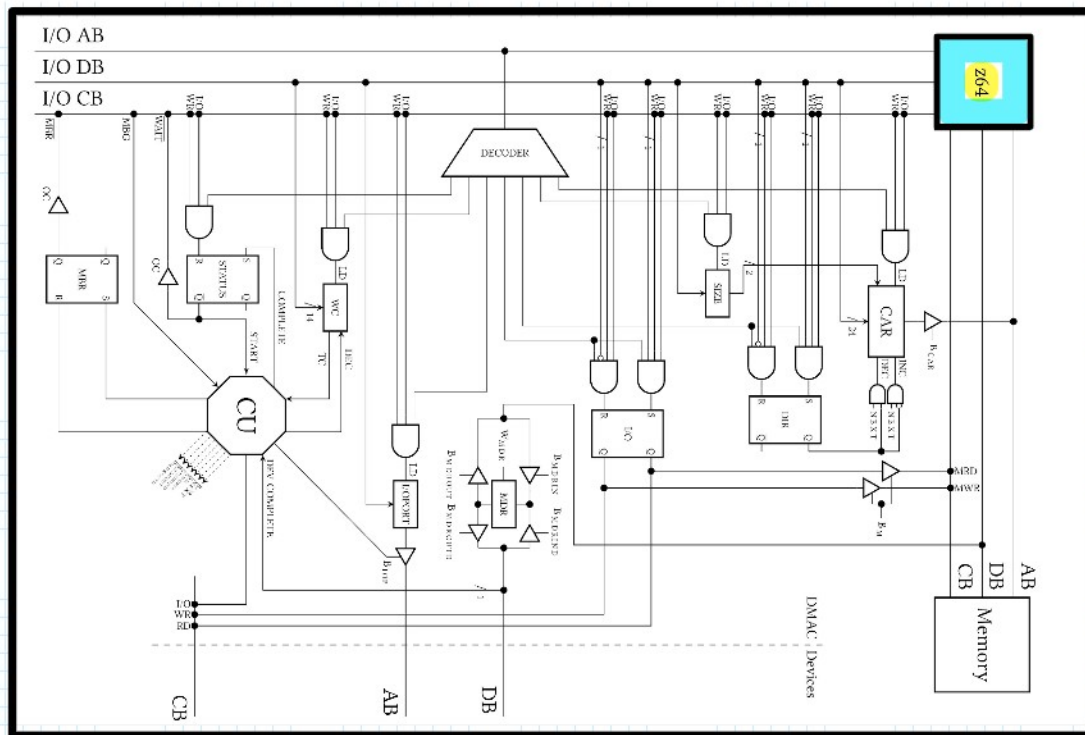
QUANTI DATI VORRANNO MESSI IN MEMORIA, LO SPECIFICO PUNTO CUI DEVONO ESSERE SCRITTI QUESTI DATI LO CHIAMIAMO TRAMITE L'INDIRIZZO!

DOBBIAMO DIRE ANCHE LA GRANDEZZA DI QUESTI DATI DA RECUPERARE DA UN CERTO DISPOSITIVO!

INTERAMENTE IL DMAC DEVE POTER GESTIRE QUESTE INFORMAZIONI DEVE ESSERE IN GRADO DI ESSERE PROGRAMMATO DALLA CPU, DEVE ESSERE IN GRADO DI TENERE TRACCA DI DOVE STA SCRIVENDO IN MEMORIA E DA DOVE STA LEGGENDO IN MEMORIA, DEVE ESSERE IN GRADO DI GESTIRE UNA CERTA TABELLA DI TRASFERIMENTI (BYTE, LONGWORD, ...) PERCHE' I DISPOSITIVI POSSANO OPERARE CON QUESTE MODALITA', E DEVE PARLARE IL LINGUAGGIO DEL MEMORY BUS.

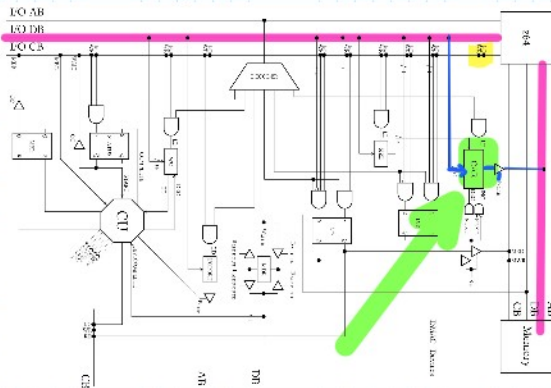
PROVIAMO A VEDERE L'INTERFACCIA CON IL DMAC!

Interfaccia DMAC-processore



DEVE ESSERE UN'INTERFACCIA SIA VERSO IL PROCESSORE SIA VERSO LA MEMORIA. Perché il mio DMAC deve scrivere o leggere dati dalla memoria; MA DEVE ANCHE POTER PARLARE CON I DISPOSITIVI CON IL BUS I/O.

CAR = È UN REGISTRO D'INTERFACCIA, ed è il registro che dice al DMAC la prossima operazione verso la memoria, a quale indirizzo è associata.



Qual'è l'indirizzo di memoria che deve essere coinvolto in una determinata operazione di memoria;

CHI SARA' A PROGRAMMARE QUESTO REGISTRO?

IL PROCESSORE!

DAL DATABUS DEL PROCESSORE ARRIVA UN SEGNALE che mi permette di scrivere all'interno del **CAR**.

Quindi se io utilizzo un'istruzione per andare a caricare il valore del CAR, il mio processore sta scrivendo l'indirizzo in cui bisogna andare ad effettuare il trasferimento.

ORA il mio DMAC prende i dati da un dispositivo e li deve scrivere a quell'indirizzo.

il contenuto di CAR deve **OVVIAMENTE** ANDARE VERSO la memoria, precisamente tramite un Buffer - Three State, verso l'ADDRESS BUS della memoria!

IL DMAC deve anche dire al nostro dispositivo per quale motivo sta andando ad effettuare un'interazione con la memoria, ossia se è un'interazione **IN LETTURA O IN SCRITTURA**. IL DMAC deve parlare la lingua del memory BUS.

Quindi significa che il mio DMAC deve gestire i segnali di Read e Write verso la memoria, MA È il processore che decide se il DMAC opera in lettura o in scrittura, quindi tutti segnali provengono dal processore;

Quindi il processore setterà quei flag a 0 o 1, a seconda se il mio DMAC debba leggere o scrivere da/verso un certo indirizzo che è scritto all'interno del CAR.

Quindi il processore setterà quel flag a 0 o 1, a seconda se il mio DMAC debba leggere o scrivere da/verso un certo indirizzo che è scritto all'interno del CAR.

Immaginiamo che il DMAC abbia scritto un dato verso la memoria, il trasferimento successivo a che indirizzo dovrà essere fatto? **UNCERTO SPOSTAMENTO A PARTIRE DELL'INDIRIZZO INIZIALE.**

HA SENSO DI CHIEDERE AL PROCESSORE DI FARE LA SCRITTURA DENTRO IL CAR? **NO!**

DEVE ESSERE IL DMAC che calcola, in qualche modo, l'indirizzo successivo!

QUINDI CAR È UN REGISTRO CHE PUÒ ESSERE INCREMENTATO!

Può essere incrementato per puntare all'area di memoria successiva;

Di quanto lo dobbiamo incrementare?

CHI DECIDE SE BISOGNA SCRIVERE A BLOCCHI DI BYTE, LONGWORD, QUADWORD?
IL PROCESSORE.

Quindi esiste un registro di interfaccia chiamato "size" che è scritto dal processore che

comunica, al DMAC se sta scrivendo alla grandezza del byte, della word, della longword o della quadword.

Quando si riceve un segnale di incremento del CAR, esso viene incrementato pari alla taglia della dimensione del singolo blocco.

QUANTI BLOCCHI DEVO COPIARE?

Però dobbiamo prevedere un altro registro WC

(WORD COUNT) CHE SERVE A CONTARE IL NUMERO DI PAROLE DI DIMENSIONE SIZE. CHE IL MIO DMAC DEVE TRASFERIRE DA/VERSO la memoria, A PARTIRE dal CAR. Il contenuto di questo registro WC viene, di nuovo, programmato dal processore, con un'operazione di ADD.

COME FA IL DMAC A CAPIRE CHE HA FINITO UN CERTO TRASFERIMENTO?
QUAND'È CHE HA FINITO DI TRASFERIRE?

QUANDO HA TRASFERITO WORD COUNT (WC) PAROLE! ✓

WORD COUNT HA UN SEGNALE che lo fa decrementare ogni volta di 1!

IN OUTPUT DA WC AVRÀ UN SEGNALE TC (TERMINAL COUNT) che è lo ZFLAG di quello specifico registro: quando il contenuto di WC si è azzerato, quel segnale vale 1.

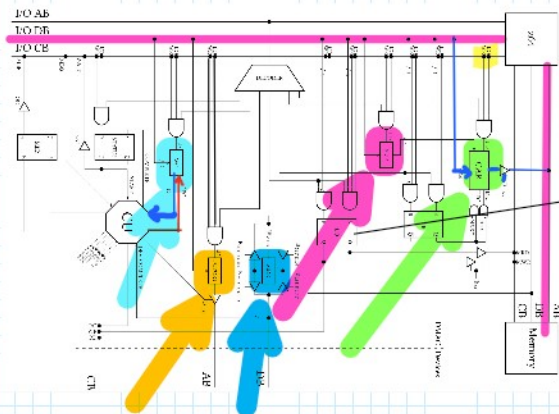
QUESTO SEGNALE VA IN INPUT ALLA CONTROL UNIT: L'UNITÀ DI CONTROLLO A QUEL PUNTO SA CHE HA FINITO DI EFFETTUARE IL TRASFERIMENTO.

Quante parole di una certa taglia, devono essere trasferite verso una determinata area di memoria, e se il trasferimento è IN INPUT o IN OUTPUT.

quante parole di una certa lingua, devono essere trasferite verso una determinata area di memoria, e se il trasferimento è in INPUT o in OUTPUT.

MA LA CPU SCEGLIE QUAL'E' IL DISPOSITIVO CON CUI SI INTERFACCIA IL DMAC!

SIGNIFICA CHE ABBIAMO BISOGNO DI UN ALTRO REGISTRO D'INTERFACCIA CHE SI CHIAMA I/O PORT;



IN quel registro, il processore scrive su quel registro l'I/O port che identifica il dispositivo con cui il DAC deve interfacciare IN INPUT O IN OUTPUT;

Quindi questo Flipflop di I/O serve anche per permettere AL DAC di mandare, sul CONTRA BUS VERSO I dispositivi, informazioni del TIPO: **INNA UN dato, devi prendere un dato.**

Ora che il DHCP ha l'io port, al centro c'è un registro MDR. Il famoso memory data register per consentire il **trasferimento bidirezionale dei dati**!

Quindi se sta eseguendo un'operazione di input, permetterà al dispositivo che corrisponde ad un altro I/O per, di scrivere i dati all'interno, se è un'operazione di output il contenuto di questo registro verrà scritto verso il data bus del dispositivo.

il contenuto di questo registro deve andare verso il database della memoria sia in input che in output.

MDR ha l'interfacciamento verso la memoria e verso i dispositivi di I/O!

Quindi diventa un registro Jampon FRA il BUS di memoria e il BUS dispositivi, e ne consente l'interfacciamento.

Quindi la CPU programma il DMA per interagire con un altro dispositivo (L'I/O port) con una certa azione di memoria (CAR) per scrivere un insieme di dati (WC) di dimensione ciascuno size, in input e in output, e a questo punto cosa dice la CPU al DMA?

VAI, FAI IL TRASFERIMENTO.

Per fare questo uso il `fprintf` di `status!` che permette di dire al processore "oh ho programmato tutti i registri di interfaccia, ora parti"

E l'UNITÀ di controllo legge questi segnali.

L'output del sleep loop di status lo attacchiamo al segnale di WAIT, comunichiamo al processore che il trasferimento non è completato: il processore si ferma, ma altri programmi possono essere mandati in esecuzione su quello specifico processore;

Ma magari quest'altro programma potrebbe voler leggere una variabile dalla memoria.

Chi determina se il processore può usare la memory bus? IL PROTOCOLLO DI HANDSHAKING, BUS STEALING.

CHI DECIDE SE UN TRASFERIMENTO DMAC DEBBA ESSERE ESEGUITO IN BUS STEALING? IL PROCESSORE!

Quindi esiste un altro flip flop di interfaccia, e questo flip flop di interfaccia (che nel disegno non c'è) serve a comunicare se deve operare in BUS STEALING o IN BUS.

↳ AL DMAC

IL FLIP FLOP MBR È LA **BUFFERIZZAZIONE** della MIA BUS Request (Memory Bus Request), **ovvero il segnale che serve a richiedere alla CPU l'uso del BUS.**

IL FLIP FLOP DIR Governa il modo in cui viene aggiornato il contenuto del CAR, Dista per direction. CAR può essere incrementato/decrementato. A seconda che questo flip flop direction valga zero o uno io incrementerò o decrementerò il contenuto del CAR.

Io posso fare un trasferimento da memoria a memoria per operare dall'inizio o dalla fine di un buffer. La stessa cosa vale per un trasferimento dispositivo - memoria.

IO SE VOLESSI PROGRAMMARE IL DMAC CONTROLLER A HAND, IO DOVREI SCRIVERE TUTTE QUESTE INFORMAZIONI CON UN'ISTRUZIONE DI OUT, MA STIAMO PARLANDO DI PROCESSORI CISC E POSSIAMO INVENTARCI UNA NUOVA ISTRUZIONE PER PROGRAMMARE IL DMAC.

ESISTONO ISTRUZIONI INS E OUTS!

- Il DMAC può essere programmato con delle istruzioni dedicate

Instruction	Syntax	Semantics
Inbound transfer of a data string	insX	Transfer an arbitrarily large buffer of data from a device.
Outbound transfer of a data string	outsX	Transfer an arbitrarily large buffer of data to a device.

- Sono istruzioni di tipo stringa

insX: leggi 10 byte da DEV

outsX: scrivi 10 byte su DEV

```
1 movq $10, %rcx
2 movq $dev, %rdi
3 movq $dev_name, %dx
4 clb
5 ins
```

```
1 movq $10, %rcx
2 movq $dev, %rdi
3 movq $dev_name, %dx
4 clb
5 outsb
```

DI PAROLE
DA TRASFERIRE

INDICAZIONE
DEST.
DELLA
MEMM.

↳ I/O PORT del dispositivo con cui voglio parlare.

CLD: Dico che voglio fare un trasferimento in avanti;

INSb: copia a blocchi di byte.

IL MICROCODICE ASSOCIATO ALL'ESECUZIONE DI INSX SARÀ:

- COPIERÀ RCX NEL WC
- COPIERÀ DX NEL REGISTRO I/O PORT
- COPIERÀ RDI NEL CAR
- IL VALORE DEL FLAG DF VERRÀ SCRITTO NEL FLIP FLOP DIRECTION DEL DMAC.

IN FLAGS.

- b, VERSION SCRITTO NEL FLIP END SIZE (1 singolo byte)